



IoT y Sistemas Embebidos

Proyecto final

Sistema de control de stock con QR, MQTT, LCD y FSM

Docentes: Nicolas Calarco, Pablo Alonso

Estudiantes: Grupo 1: Victoria Amor, Nicolas Meyer y Bruno Moreira
Grupo 2: Santiago Blanco, Felipe Paladino y Piero Saucedo
Grupo 3: Angelo Gauthier, Pilar Martínez e Ignacio Negri
Grupo 4: Conrado Canzani, Emiliano Brasesco

2 de Julio de 2026

Índice

1. Introducción	2
2. Arquitectura general del sistema	2
2.1. Responsabilidades por módulo	2
2.2. Roles de dispositivo	3
2.3. Arquitectura orientada a eventos	3
3. Comunicación MQTT, Logger y NTP	4
3.1. Objetivo del módulo	4
3.2. Objetivos específicos	5
3.3. Flujo de inicialización y publicación	5
3.4. Tópicos MQTT	6
3.5. Formato JSON de los mensajes	7
3.6. Estructuras compartidas	7
3.7. Logger de eventos	8
3.8. Persistencia del logger mediante NVS	10
3.9. Sincronización horaria con NTP	10
4. FSM, product_db, hash_table y NVS	11
4.1. Introducción y objetivo	11
4.2. Máquina de estados finitos	11
4.3. Flujo completo de un QR válido	12
4.4. Manejo de errores y timers	13
4.5. Callbacks de la FSM	13
4.6. Base local de productos: product_db	14
4.7. Relación entre product_db y hash_table	15
4.8. hash_table	16
4.9. NVS y cola pendiente persistente	16
4.10. Inicialización en app_main	17
5. WiFi Manager, HTTP Handler, LCD Port y LCD Manager	18
5.1. Objetivo del grupo HTTPS/LCD	18
5.2. WiFi Manager	18
5.3. HTTP Handler	19
5.4. LCD Port	19
5.5. LCD Manager	20
6. QR Handler, cámara y touchpad	21
6.1. QR Handler y cámara	21
6.2. Configuración de hardware de cámara	21
6.3. Flujo de la tarea de escaneo	22

6.4. Touchpad	22
6.5. Calibración y detección de toques	23
6.6. Tarea de monitoreo e integración con la FSM	23
7. Dashboard y telemetría	23
8. Integración completa del sistema	25
8.1. Orden de inicialización	25
8.2. Transcurso general del código	26
9. Pruebas y validación	27
10. Problemas encontrados y decisiones tomadas	28
11. Conclusiones	28
12. Trabajo futuro	29
13. Referencias	29
A. Anexo: funciones principales	30

1. Introducción

El proyecto consiste en desarrollar un sistema de control de stock basado en el escaneo de códigos QR utilizando dos placas ESP32-S2. La primera placa, asociada a la cámara, se encarga de leer el código QR del producto y enviar la información obtenida mediante MQTT. Esa información incluye los datos del producto leído, el timestamp del escaneo y el estado de la operación.

La segunda placa recibe el mensaje por MQTT y muestra la información en la pantalla LCD. A partir de esta interfaz, el usuario puede interactuar con el sistema mediante un touchpad con botones de navegación, utilizados para desplazarse entre opciones, confirmar acciones e ingresar la cantidad de stock correspondiente.

Si el producto escaneado ya existe en la base de datos, la placa con LCD permite confirmar si se desea modificar su stock e ingresar la cantidad correspondiente. Si el QR no corresponde a un producto registrado, el sistema informa el error y evita modificar la base local. Esta decisión permite mantener controlado el catálogo y evita que lecturas no válidas alteren el inventario.

El documento fue reorganizado para eliminar repeticiones entre secciones. Por eso, algunos conceptos que aparecían varias veces en el informe original, como los roles de placa, los callbacks de la FSM y la cola de eventos, se explican una sola vez y luego se referencian desde los módulos correspondientes.

2. Arquitectura general del sistema

2.1. Responsabilidades por módulo

La arquitectura del proyecto se dividió en componentes independientes. Esta separación permite que cada grupo trabaje sobre una responsabilidad clara y que la integración se realice mediante interfaces definidas.

Módulo o grupo	Responsabilidad principal
MQTT, Logger y NTP	Comunicación entre placas, publicación de eventos, registro local, timestamps y preparación de telemetría para dashboard.
FSM, <code>product_db</code> , NVS y <code>hash_table</code>	Lógica central del sistema, transición entre estados, base local de productos, persistencia y estructuras de datos.
WiFi, HTTP y LCD	Portal de configuración WiFi, inicialización de pantalla, capa de presentación y feedback visual.
QR y Touchpad	Captura y decodificación de QR mediante cámara, y generación de eventos de usuario mediante sensores táctiles.

Cuadro 1: Responsabilidades principales dentro del proyecto.

2.2. Roles de dispositivo

El sistema usa dos placas físicas. El mismo proyecto puede compilarse para uno de los dos roles usando macros definidas en `device_role.h`:

- `DEVICE_IS_CAM`: placa con cámara. Lee códigos QR y publica eventos al broker MQTT.
- `DEVICE_IS_LCD`: placa con pantalla LCD y touchpad. Recibe información por MQTT, consulta la base local, permite confirmar cantidades y publica el stock actualizado.

Esta división fue necesaria porque no tenía sentido que ambas placas levantaran todos los componentes. La placa de la cámara no necesita pantalla, touchpad ni FSM principal; su tarea es detectar QR y comunicar el resultado. En cambio, la placa LCD sí necesita la FSM completa, porque es la encargada de confirmar productos, elegir cantidades, actualizar stock, mostrar mensajes y publicar el resultado final.

2.3. Arquitectura orientada a eventos

El sistema sigue un principio basado en eventos. Los módulos periféricos producen eventos y los envían a una cola común llamada `fsm_event_queue`. La FSM consume esos eventos uno por uno y decide qué transición ejecutar.

```
fsm_event_queue = xQueueCreate(20, sizeof(EventType));
```

Listing 1: Creación de la cola de eventos de la FSM.

El tamaño de la cola permite almacenar hasta 20 eventos pendientes. Esto es útil porque los eventos pueden venir de distintas fuentes: MQTT, touchpad, WiFi, temporizadores

internos o simulación en modo demo. Al centralizar el procesamiento en la FSM, se evita que varios módulos modifiquen el flujo al mismo tiempo.

Módulo externo → evento → fsm_event_queue → fsm_task → transición
 FSM → acción asociada → nuevo estado

3. Comunicación MQTT, Logger y NTP

3.1. Objetivo del módulo

El módulo de comunicación tuvo como objetivo desarrollar la capa de intercambio de información entre placas, el registro local de eventos, la permanencia de ese historial mediante NVS, la incorporación de timestamps mediante NTP y la preparación de datos para su visualización en un dashboard.

La solución se estructuró en componentes separados: `mqtt_handler`, `logger`, `nvs`, `ntp_handler` y `shared`. Esta separación facilita la reutilización del código, permite probar cada bloque por separado y deja una arquitectura clara para la integración con el resto del sistema.

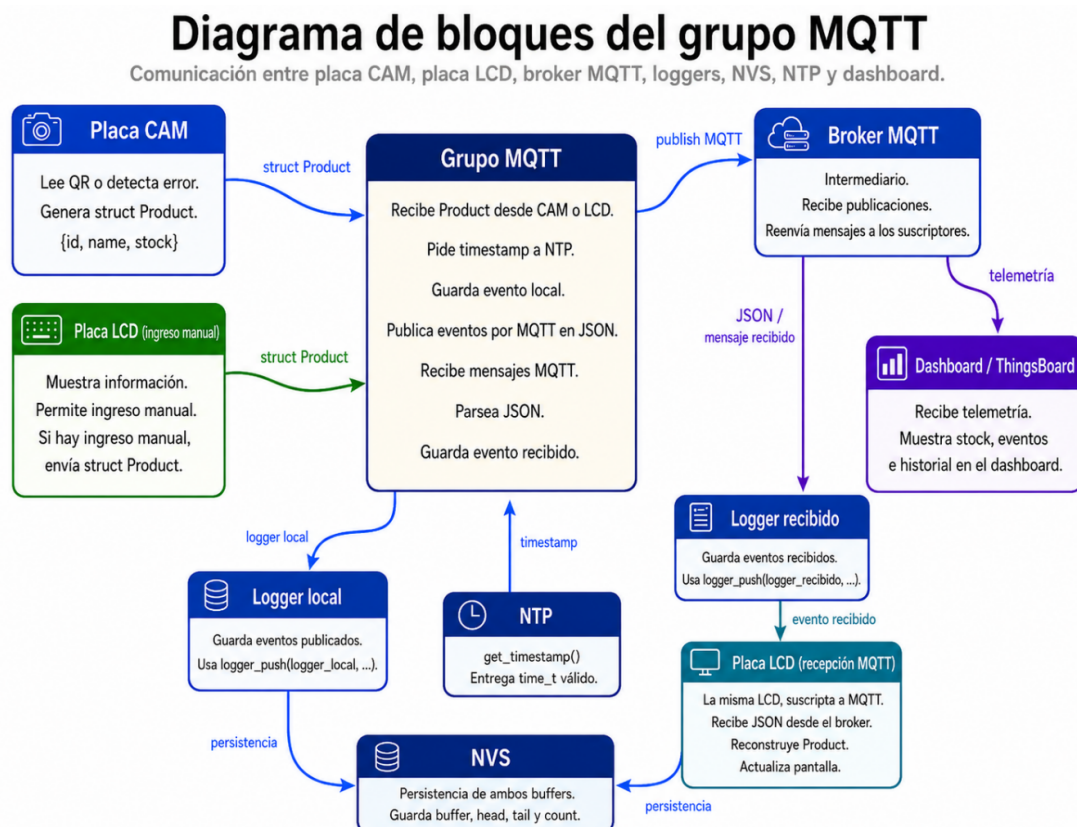


Figura 1: Arquitectura general de comunicación entre la placa CAM, la placa LCD, el broker MQTT, el logger y el dashboard.

3.2. Objetivos específicos

- Definir una estructura común para representar productos y eventos dentro del sistema.
- Publicar por MQTT los eventos generados por la placa CAM, como lecturas de QR o errores.
- Publicar por MQTT eventos generados desde la placa LCD, como actualizaciones de stock o errores.
- Recibir mensajes MQTT enviados por otra placa mediante el broker.
- Reconstruir la información recibida a partir de mensajes JSON.
- Evitar que una placa registre como recibido un mensaje que ella misma publicó.
- Guardar eventos publicados y recibidos en loggers separados.
- Conservar registros mediante NVS para no perderlos al reiniciar.
- Asociar cada evento a un tiempo usando NTP y el tipo `time_t`.
- Preparar la información para su posterior uso en ThingsBoard.

3.3. Flujo de inicialización y publicación

La arquitectura implementada empieza con una etapa de inicialización desde `app_main`. En esta etapa se inicializan los loggers mediante `logger_init`, creando un logger para eventos locales y otro para eventos recibidos desde MQTT. Luego, estos loggers se entregan al módulo MQTT con `mqtt_handler_set_loggers`, de forma que el módulo pueda guardar eventos tanto al publicar como al recibir mensajes.

También se inicializa la hora con `init_time`, necesaria para obtener timestamps válidos mediante NTP. Finalmente se llama a `iniciar_mqtt`, que crea el cliente MQTT, registra el manejador de eventos y deja la placa lista para conectarse, publicar y recibir mensajes.

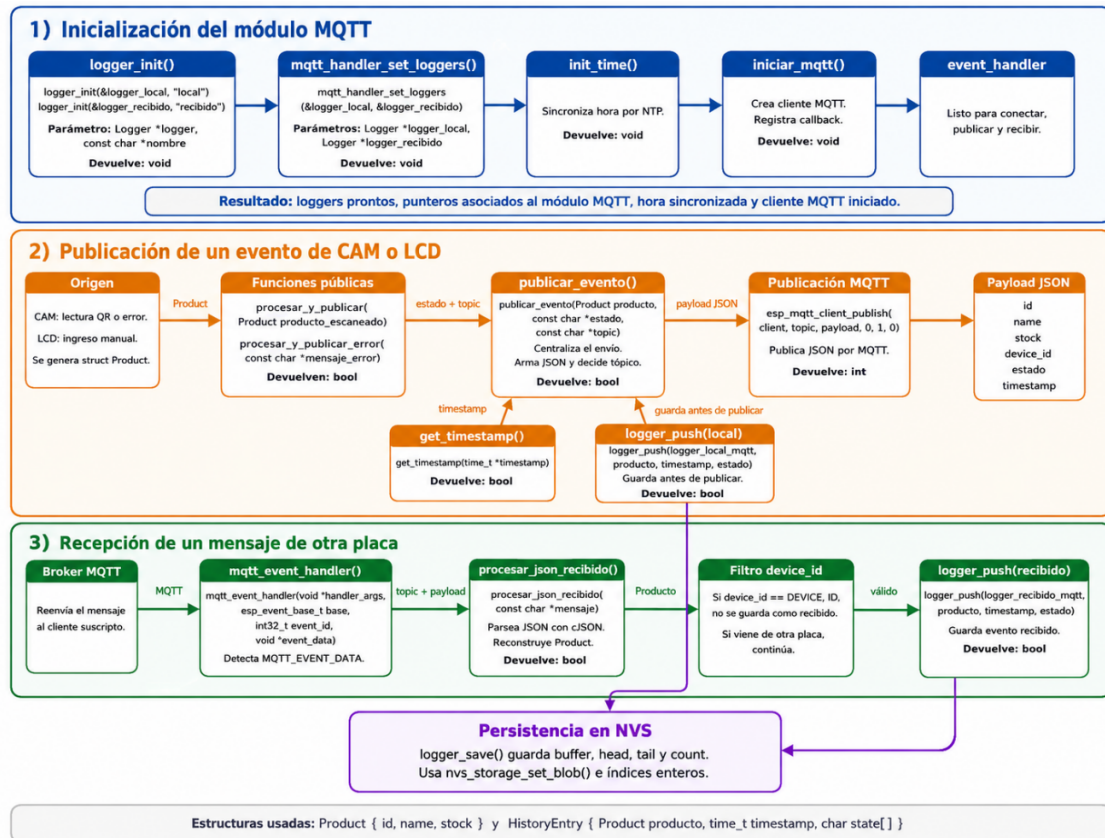


Figura 2: Flujo interno desde app_main hasta la publicación, recepción y persistencia de eventos.

Cuando se desea publicar un evento, el flujo converge en una función central de publicación. Esta función recibe el producto, el estado del evento y el tópico correspondiente. Primero solicita un timestamp válido con `get_timestamp`; luego guarda el evento en el logger local usando `logger_push`; después construye el payload JSON y llama a `esp_mqtt_client_publish` para publicar el mensaje en el tópico correspondiente.

Guardar primero en el logger y recién después intentar publicar fue una decisión importante. Si la red falla o el broker no está disponible, el evento queda registrado localmente y no se pierde el historial inmediato.

3.4. Tópicos MQTT

El sistema utiliza tópicos separados según el objetivo de la comunicación.

Tópico	Uso
ucuiot/magno/x7k2/scan	Canal principal entre placas. La placa CAM publica productos escaneados y la placa LCD los recibe.
ucuiot/magno/x7k2/telemetry	Canal de telemetría para enviar stock actualizado hacia la integración con ThingsBoard.

Cuadro 2: Tópicos principales usados por el sistema.

En la versión revisada del informe se omitió la mención dudosa al tópico `control/configuracion`, ya que aparecía como comentario no confirmado en el documento original. El flujo validado para el proyecto se concentra en los tópicos `/scan` y `/telemetry`.

3.5. Formato JSON de los mensajes

Para transmitir la información se eligió JSON porque permite enviar varios campos dentro de un payload de texto legible y fácil de reconstruir del lado receptor. Cada mensaje incluye un identificador de dispositivo, el id del producto, el nombre, el stock, el timestamp y el estado.

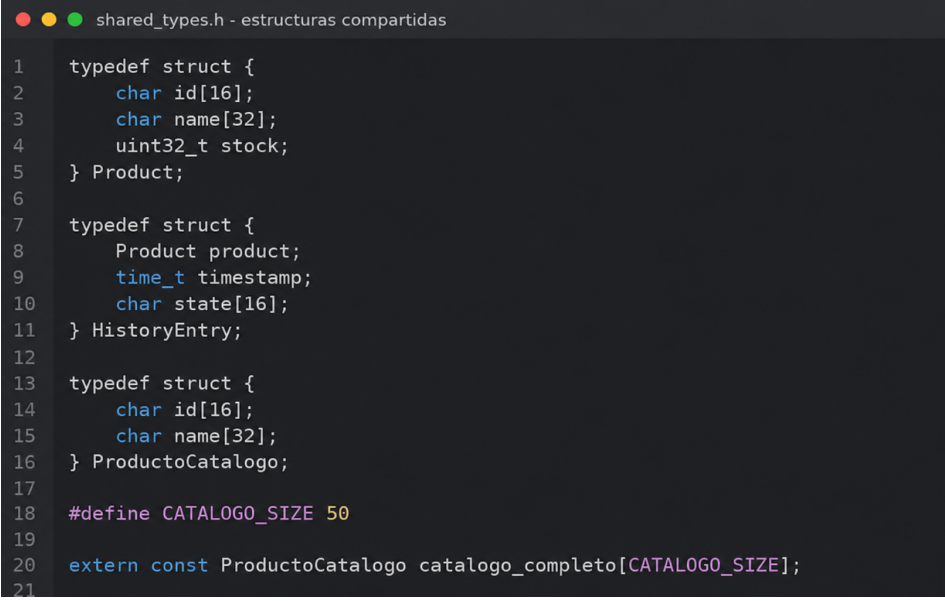
```
{
  "device_id": "LCD_01",
  "id": "PROD001",
  "producto": "Leche",
  "stock": 12,
  "timestamp": 1783000000,
  "estado": "OK"
}
```

Listing 2: Ejemplo conceptual del payload JSON.

Cuando el mensaje llega a la placa suscripta, se utiliza `cJSON` para parsearlo, comprobar el tipo de cada campo y reconstruir la estructura `Product`. Además, se compara el `device_id` recibido con el identificador local para evitar que una placa registre como recibido un mensaje que ella misma publicó.

3.6. Estructuras compartidas

Para que todos los componentes trabajaran con el mismo formato, se definieron estructuras compartidas. `Product` representa al producto y `HistoryEntry` representa un evento guardado en el historial.



```

1  typedef struct {
2      char id[16];
3      char name[32];
4      uint32_t stock;
5  } Product;
6
7  typedef struct {
8      Product product;
9      time_t timestamp;
10     char state[16];
11 } HistoryEntry;
12
13 typedef struct {
14     char id[16];
15     char name[32];
16 } ProductoCatalogo;
17
18 #define CATALOGO_SIZE 50
19
20 extern const ProductoCatalogo catalogo_completo[CATALOGO_SIZE];
21

```

Figura 3: Estructuras compartidas definidas en `shared_types.h`.

Esta decisión hace que el logger no dependa del origen del dato, ya que puede almacenar eventos escaneados, ingresos manuales, errores o mensajes reconstruidos desde MQTT.

3.7. Logger de eventos

El logger se diseñó como un buffer circular de tamaño fijo. Cada posición del buffer almacena una entrada `HistoryEntry`, que contiene el producto, el timestamp y el estado del evento. Para controlar el recorrido del buffer se utilizan tres variables:

- **head**: posición donde se encuentra el evento más antiguo.
- **tail**: próxima posición donde se va a escribir.
- **count**: cantidad de eventos válidos almacenados.

```

1  bool logger_push(Logger *logger, Product product, time_t timestamp, const char *state)
2  {
3      if (logger == NULL || timestamp == 0 || state == NULL) {
4          return false;
5      }
6
7      memset(&logger->buffer[logger->tail], 0, sizeof(logger->buffer[logger->tail]));
8
9      logger->buffer[logger->tail].product = product;
10     logger->buffer[logger->tail].timestamp = timestamp;
11
12     strncpy(logger->buffer[logger->tail].state,
13             state,
14             sizeof(logger->buffer[logger->tail].state) - 1);
15
16     logger->buffer[logger->tail].state[
17         sizeof(logger->buffer[logger->tail].state) - 1] = '\0';
18
19     logger->tail = (logger->tail + 1) % LOGGER_SIZE;
20
21     if (logger->count < LOGGER_SIZE) {
22         logger->count++;
23     } else {
24         logger->head = (logger->head + 1) % LOGGER_SIZE;
25         ESP_LOGW(TAG,
26                 "Logger %s lleno: se sobrescribio el evento mas viejo",
27                 logger->nombre);
28     }
29
30     logger_save(logger);
31
32     ESP_LOGI(TAG,
33             "Guardado en logger %s: %s | %s | %lu | %lld | %s",
34             logger->nombre,
35             product.id,
36             product.name,
37             (unsigned long)product.stock,
38             (long long)timestamp,
39             state);
40
41     return true;
42 }

```

Figura 4: Fragmento del módulo logger.c.

Cada vez que se agrega un evento con `logger_push`, se escribe en la posición indicada por `tail`. Luego, `tail` se actualiza con la operación `(tail + 1) % LOGGER_SIZE`. Cuando el buffer está lleno, el nuevo evento sobrescribe al más antiguo y también se actualiza `head`.

Funcionamiento del buffer circular del logger

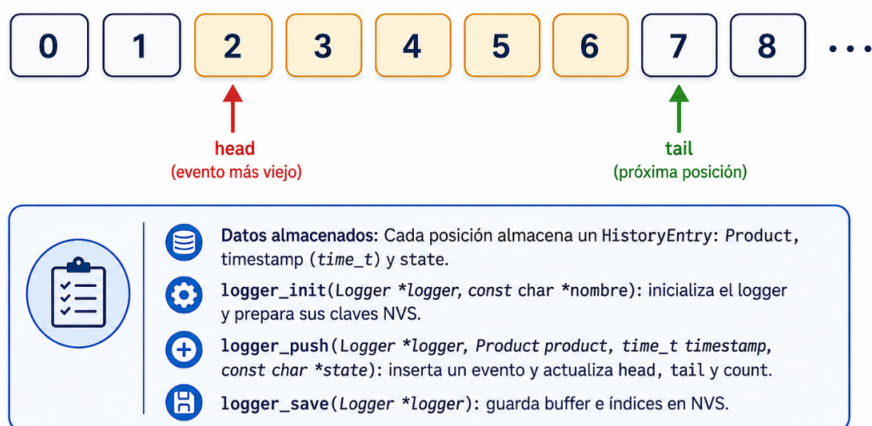


Figura 5: Funcionamiento circular del buffer del logger.

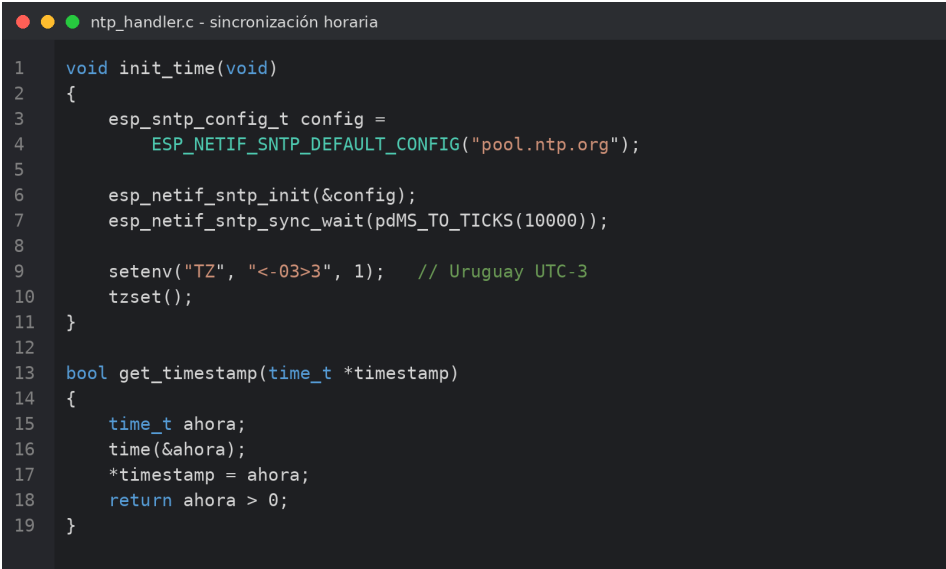
3.8. Persistencia del logger mediante NVS

Con el objetivo de que el historial no se pierda al reiniciar la placa, el logger se integró con NVS. El buffer completo se guarda como un blob, mientras que `head`, `tail` y `count` se guardan como enteros. Esto es necesario porque, si solo se guardaran los eventos pero no los índices, al reiniciar la placa no se sabría cuál era el evento más viejo, cuál era la próxima posición libre ni cuántos eventos válidos había almacenados.

Durante la inicialización, `logger_init` intenta recuperar el buffer guardado mediante `nvs_storage_get_blob` y los índices mediante `nvs_storage_get_int`. Si los datos existen y son válidos, el logger continúa desde el estado anterior. Si no hay datos guardados o los índices están fuera de rango, el logger se reinicia vacío.

3.9. Sincronización horaria con NTP

Para ordenar temporalmente los eventos se implementó el módulo `ntp_handler`. La función `init_time` configura la sincronización SNTP utilizando el servidor `pool.ntp.org`. Luego de sincronizar, se configura la zona horaria de Uruguay, UTC-3.



```
1 void init_time(void)
2 {
3     esp_sntp_config_t config =
4         ESP_NETIF_SNTP_DEFAULT_CONFIG("pool.ntp.org");
5
6     esp_netif_sntp_init(&config);
7     esp_netif_sntp_sync_wait(pdMS_TO_TICKS(10000));
8
9     setenv("TZ", "<-03>3", 1); // Uruguay UTC-3
10    tzset();
11 }
12
13 bool get_timestamp(time_t *timestamp)
14 {
15     time_t ahora;
16     time(&ahora);
17     *timestamp = ahora;
18     return ahora > 0;
19 }
```

Figura 6: Obtención y validación del timestamp en `ntp_handler.c`.

Una vez realizada esta etapa, el sistema puede obtener la hora actual mediante `get_timestamp(time_t *timestamp)`. El tipo `time_t` representa la cantidad de segundos transcurridos desde el 1 de enero de 1970, por lo que resulta práctico para comparar, guardar y publicar eventos.

4. FSM, `product_db`, `hash_table` y NVS

4.1. Introducción y objetivo

La responsabilidad principal de este grupo fue desarrollar la lógica central del sistema: la máquina de estados finitos, la base local de productos, la estructura de datos utilizada para búsquedas rápidas y la capa de almacenamiento persistente mediante NVS.

Dentro de la arquitectura general, la FSM cumple el rol de coordinador de la placa LCD. No lee directamente la cámara, no maneja pines de la pantalla y no publica por MQTT de forma directa. En cambio, recibe eventos ya procesados desde otros módulos, consulta la base de productos, decide qué transición corresponde, actualiza el stock cuando el usuario confirma una operación y solicita a otros componentes que muestren información o publiquen datos.

4.2. Máquina de estados finitos

La FSM está implementada en `fsm.c` y `fsm.h`. Su objetivo es representar formalmente los estados posibles del sistema y las transiciones válidas entre ellos.

Estado	Descripción
STATE_IDLE	Estado inicial. El sistema espera un QR.
STATE_SCAN_PROCESSING	Se recibió un QR y se valida si contiene información útil.
STATE_LOCAL_DB_LOOKUP	Se busca el producto en la base local.
STATE_PROMPT_ADD_PRODUCT	Se muestra el producto reconocido y se espera confirmación del usuario.
STATE_QUANTITY_SELECTION	El usuario selecciona la cantidad a agregar con el touchpad.
STATE_STOCK_UPDATING	Se actualiza el stock en <code>product_db</code> .
STATE_DISPLAY_PRODUCT_INFO	Se muestra el resultado de la operación.
STATE_MQTT_PUBLISHING	Se solicita publicar el producto actualizado por MQTT.
STATE_ERROR_DISPLAY	Se muestra un error y luego se vuelve a STATE_IDLE.

Cuadro 3: Estados principales de la FSM.

Los eventos principales de la FSM son los siguientes:

Evento	Origen o significado
EV_QR_CAPTURED	Cámara, MQTT o demo cuando llega un QR decodificado.
EV_SCAN_SUCCESS	Evento interno generado si el QR contiene un producto válido.
EV_SCAN_INVALID	Evento interno generado si el QR está vacío o corrupto.
EV_PRODUCT_FOUND	Evento interno generado si <code>product_db</code> encuentra el producto.
EV_PRODUCT_NOT_FOUND	Evento interno generado si el producto no existe en el catálogo local.
EV_BTN_UP	Touchpad. Aumenta la cantidad seleccionada.
EV_BTN_DOWN	Touchpad. Disminuye la cantidad seleccionada.
EV_BTN_CONFIRM	Touchpad. Confirma la acción actual.
EV_BTN_CANCEL	Touchpad. Cancela la operación actual.
EV_STOCK_UPDATED	Evento interno generado después de actualizar el stock.
EV_MQTT_PUBLISH_SUCCESS	MQTT aceptó la publicación.
EV_MQTT_PUBLISH_FAILURE	MQTT no pudo publicar.
EV_TIMEOUT	Timer interno usado para mantener pantallas temporales.

Cuadro 4: Eventos principales de la FSM.

La FSM no se escribió como una serie de `if` anidados, sino como una tabla de transiciones. Cada fila de la tabla contiene el estado actual, el evento recibido, el próximo estado y la acción asociada. Esta forma de diseño deja el flujo explícito y facilita agregar comportamientos nuevos sin alterar muchas condiciones distribuidas por el código.

4.3. Flujo completo de un QR válido

El flujo normal comienza cuando llega un QR decodificado. La entrada pública para esto es `fsm_on_qr_detected(const char *id, const char *name)`. Esta función solo acepta el QR si la FSM está en `STATE_IDLE`. Si el sistema está ocupado, el QR se ignora para evitar que una lectura nueva interrumpa una operación que todavía no terminó.

El recorrido completo es:

1. `STATE_IDLE + EV_QR_CAPTURED`: la FSM pasa a `STATE_SCAN_PROCESSING` y valida

que exista un producto activo con id no vacío.

2. `STATE_SCAN_PROCESSING + EV_SCAN_SUCCESS`: se pasa a `STATE_LOCAL_DB_LOOKUP` y se consulta `product_db_find_by_id`.
3. `STATE_LOCAL_DB_LOOKUP + EV_PRODUCT_FOUND`: si el producto existe, se usa el producto guardado en la base local como fuente de verdad.
4. `STATE_PROMPT_ADD_PRODUCT`: la LCD muestra el producto reconocido y consulta si se desea agregar stock.
5. `STATE_QUANTITY_SELECTION`: el usuario ajusta la cantidad mediante `EV_BTN_UP` y `EV_BTN_DOWN`.
6. `STATE_STOCK_UPDATING`: al confirmar, la FSM llama a `product_db_add_stock`.
7. `STATE_DISPLAY_PRODUCT_INFO`: se muestra el producto actualizado con el nuevo stock.
8. `STATE_MQTT_PUBLISHING`: se solicita la publicación MQTT mediante el callback correspondiente.
9. `STATE_IDLE`: si la publicación fue aceptada, el sistema vuelve al estado de espera. Si falló, intenta guardar el evento como pendiente y también vuelve a esperar.

IDLE → QR capturado → validación → búsqueda en `product_db` →
confirmación → selección de cantidad → actualización de stock →
publicación MQTT → IDLE

4.4. Manejo de errores y timers

Si el QR está vacío, corrupto o no contiene un id válido, la FSM guarda el motivo en `last_error` y genera `EV_SCAN_INVALID`. Luego pasa a `STATE_ERROR_DISPLAY`, donde muestra el error y, si existe callback registrado, publica el error por MQTT.

Si el QR es válido pero el producto no existe en el catálogo local, `lookup_product` genera `EV_PRODUCT_NOT_FOUND`. En la versión actual se decidió no abrir un flujo de ingreso manual para productos fuera de catálogo, para evitar que se agreguen productos no controlados por el catálogo compartido.

La FSM utiliza dos timers de FreeRTOS: `s_timeout_timer` y `s_idle_screen_timer`. El primero mantiene visibles pantallas temporales, como errores o confirmaciones, durante `FSM_DISPLAY_HOLD_MS` (2500 ms). El segundo vuelve a mostrar la pantalla de espera después de regresar a `STATE_IDLE`, usando `FSM_IDLE_SCREEN_DELAY_MS` (800 ms).

4.5. Callbacks de la FSM

Uno de los aspectos más importantes del diseño fue el uso de callbacks. La FSM no llama directamente a `lcd_show_product`, `procesar_y_publicar` ni `pending_queue_push`. En

cambio, define una estructura `FsmCallbacks` con punteros a funciones que se registran desde `app_main`.

Callback	Responsabilidad
<code>show_waiting</code>	Mostrar la pantalla de espera.
<code>show_product_prompt</code>	Mostrar el producto reconocido.
<code>show_quantity_selection</code>	Mostrar la selección de cantidad.
<code>show_success</code>	Mostrar que el stock fue actualizado.
<code>show_cancelled</code>	Mostrar operación cancelada.
<code>show_error</code>	Mostrar un error.
<code>publish_qr</code>	Publicar el producto actualizado por MQTT.
<code>publish_error</code>	Publicar un error por MQTT.
<code>store_pending_qr</code>	Guardar un producto pendiente si falla MQTT.
<code>flush_pending</code>	Reenviar pendientes cuando MQTT vuelve a conectarse.

Cuadro 5: Callbacks principales registrados en la FSM.

Esto desacopla la FSM del hardware y de la red. La FSM solo sabe que debe mostrar o publicar algo, pero no conoce la implementación concreta. En la placa real, esos callbacks apuntan a `lcd_manager` y `mqtt_handler`; en pruebas, pueden apuntar a funciones simuladas.

```
FsmCallbacks callbacks = {
    .show_waiting          = lcd_show_waiting_cb,
    .show_product_prompt   = lcd_show_product_prompt_cb,
    .show_quantity_selection = lcd_show_quantity_selection_cb,
    .show_success          = lcd_show_success_cb,
    .show_cancelled        = lcd_show_cancelled_cb,
    .show_error            = lcd_show_error_cb,
    .publish_qr            = mqtt_publish_product_cb,
    .store_pending_qr      = mqtt_store_pending_cb,
    .flush_pending         = mqtt_flush_pending_cb,
};
```

Listing 3: Ejemplo de registro de callbacks desde el main.

4.6. Base local de productos: `product_db`

El módulo `product_db` funciona como una capa intermedia entre la FSM y la estructura de datos real. La FSM no accede directamente a la `hash_table`, sino que utiliza funciones

públicas de `product_db`.

Función	Descripción
<code>product_db_init</code>	Inicializa la tabla de productos.
<code>product_db_load_catalog</code>	Carga un catálogo recibido por parámetro.
<code>product_db_load_default_catalog</code>	Carga el catálogo compartido definido en <code>shared_types.c</code> .
<code>product_db_upsert_product</code>	Inserta un producto nuevo o actualiza uno existente.
<code>product_db_add_stock</code>	Suma una cantidad al stock de un producto existente.
<code>product_db_register_scan</code>	Registra un escaneo como suma de una unidad, por compatibilidad.
<code>product_db_find_by_id</code>	Busca un producto por id.
<code>product_db_decrement_stock</code>	Resta una unidad de stock.
<code>product_db_remove_product</code>	Elimina un producto de la tabla.
<code>product_db_get_count</code>	Devuelve la cantidad de productos cargados.

Cuadro 6: Funciones principales de `product_db`.

La base trabaja con la estructura `Product`, que contiene `id`, `name` y `stock`. Además, existe `ProductoCatalogo`, que representa los productos del catálogo inicial. En esta versión, el catálogo tiene 30 productos y se carga al arrancar la placa LCD mediante `product_db_load_default_catalog`.

Un punto importante es que el QR no define el stock. El QR solo identifica el producto. El stock se mantiene en `product_db` y cambia únicamente cuando el usuario confirma una cantidad desde la LCD.

4.7. Relación entre `product_db` y `hash_table`

`product_db` no almacena los productos en un arreglo simple, sino que utiliza internamente una tabla hash definida en el componente `dsa`. Esta tabla se declara como variable estática:

```
static HashTable s_product_table;
```

Esto significa que el resto del sistema no accede directamente a la tabla. La única forma correcta de consultar o modificar productos es usando las funciones públicas de `product_db`. Esta decisión mejora el encapsulamiento y evita que otros módulos alteren la estructura interna por error.

4.8. hash_table

La `hash_table` es la estructura de datos utilizada para almacenar y buscar productos de forma eficiente. Tiene tamaño fijo:

```
#define HASH_TABLE_SIZE 64
```

Esto es adecuado para el proyecto porque el catálogo inicial tiene 30 productos. Al usar 64 posiciones, queda margen para reducir colisiones y mantener búsquedas rápidas.

Cada posición de la tabla es un `HashEntry`, compuesto por un `Product` y un `EntryStatus`. El estado puede ser:

Estado	Significado
<code>ENTRY_EMPTY</code>	Celda vacía y nunca usada.
<code>ENTRY_OCCUPIED</code>	Celda ocupada por un producto.
<code>ENTRY_DELETED</code>	Celda que tuvo un producto, pero fue eliminada.

Cuadro 7: Estados posibles de una entrada de la tabla hash.

Para calcular la posición inicial se utiliza el algoritmo `djb2`, una función de hash simple y rápida para strings. Si ocurre una colisión, la tabla utiliza sondeo lineal: prueba la siguiente posición, luego la siguiente, y así sucesivamente hasta encontrar el producto o una celda disponible.

Esta implementación permite búsquedas, inserciones y actualizaciones eficientes en promedio, sin usar memoria dinámica. Esto es importante en sistemas embebidos, donde conviene reservar memoria fija y evitar fragmentación.

4.9. NVS y cola pendiente persistente

El módulo `nvs` funciona como una capa de abstracción sobre la librería NVS de ESP-IDF. NVS permite guardar información en la flash de la placa mediante pares clave-valor. A diferencia de la RAM, estos datos no se pierden cuando la placa se reinicia.

Función	Descripción
<code>nvs_storage_init</code>	Inicializa la partición NVS y abre el namespace.
<code>nvs_storage_clear</code>	Borra la partición NVS y la vuelve a inicializar.
<code>nvs_storage_set_str</code>	Guarda una string.
<code>nvs_storage_get_str</code>	Recupera una string.
<code>nvs_storage_set_int</code>	Guarda un entero de 32 bits.

Función	Descripción
<code>nvs_storage_get_int</code>	Recupera un entero de 32 bits.
<code>nvs_storage_set_blob</code>	Guarda un bloque de memoria completo.
<code>nvs_storage_get_blob</code>	Recupera un bloque de memoria completo.

Cuadro 8: Funciones principales del wrapper NVS.

En el proyecto, esta capa es usada por varios componentes: `wifi_manager` guarda credenciales WiFi, `logger` guarda historiales y `pending_queue` conserva eventos MQTT que no pudieron publicarse.

La FSM se integra con esta persistencia a través del callback `store_pending_qr`. Cuando la publicación MQTT falla, la FSM pasa por `STATE_MQTT_PUBLISHING`, recibe `EV_MQTT_PUBLISH_FAILURE` y llama a la función que guarda el evento en una cola pendiente.

La cola pendiente tiene tamaño fijo:

```
#define PENDING_QUEUE_SIZE 20
```

Funciona como buffer circular con `head`, `tail` y `count`. Cada evento pendiente contiene `Product`, `time_t`, estado y tópico. Para persistirla, se guarda el buffer completo como blob y los índices como enteros. Cuando MQTT vuelve a conectarse, la FSM recibe `EV_MQTT_CONNECT_SUCCESS` y ejecuta el callback `flush_pending` para reenviar los eventos guardados.

4.10. Inicialización en `app_main`

En la placa LCD, la inicialización relacionada con la FSM y `product_db` ocurre en `app_lcd_init`. El orden es:

1. Se crea `fsm_event_queue`.
2. Se inicializa la LCD mediante `lcd_port_init`.
3. Se inicializa la capa de presentación con `lcd_manager_init`.
4. Se arma la estructura `FsmCallbacks`.
5. Se inicializa `product_db`.
6. Se carga el catálogo compartido.
7. Se inicializa la FSM.
8. Se registran los callbacks.

9. Se crea la tarea `fsm_task`.
10. Se inicializa el touchpad.
11. Se crea la tarea de lectura del touchpad.

Este orden es importante: la cola debe existir antes de que cualquier módulo envíe eventos, `product_db` debe estar inicializada antes de buscar productos, y los callbacks deben estar registrados antes de que la FSM intente mostrar pantallas o publicar por MQTT.

5. WiFi Manager, HTTP Handler, LCD Port y LCD Manager

5.1. Objetivo del grupo HTTPS/LCD

Este bloque del proyecto se encargó de la conectividad local, la configuración remota y la capa de visualización. Sus componentes principales fueron `wifi_manager`, `http_handler`, `lcd_port` y `lcd_manager`.

5.2. WiFi Manager

El ESP32 opera en modo AP+STA. Esto significa que levanta su propio punto de acceso para configuración y, al mismo tiempo, intenta conectarse como estación a la red del usuario. El portal en `192.168.4.1` sigue disponible aunque el dispositivo ya esté conectado a internet.

El SSID del AP se genera dinámicamente a partir de los últimos tres bytes de la MAC, para evitar colisiones cuando hay varias placas encendidas en el mismo entorno.

```
uint8_t mac[6];
esp_read_mac(mac, ESP_MAC_WIFI_STA);
snprintf(s_ap_ssid, sizeof(s_ap_ssid),
         "ESP32-%02X%02X%02X", mac[3], mac[4], mac[5]);
```

Listing 4: Generación de SSID a partir de la MAC.

Las credenciales se guardan usando `nvs_storage`. Al limpiarlas, se pisan solo las claves `ssid` y `password`, sin borrar el namespace completo. Esto evita afectar datos de otros módulos, como `product_db`, `logger` o `pending_queue`.

```
nvs_storage_set_str("ssid", "");
nvs_storage_set_str("password", "");
```

Listing 5: Limpieza segura de credenciales WiFi.

Color LED	Significado
Blanco	La interfaz STA arrancó e intenta conectarse.
Verde	Conexión establecida e IP obtenida. Se envía EV_WIFI_CONNECT_SUCCESS a la FSM.
Rojo	Desconexión. Si hay credenciales guardadas, se reintenta automáticamente.

Cuadro 9: Feedback visual del WiFi Manager.

5.3. HTTP Handler

El `http_handler` sirve un portal de configuración WiFi desde el propio ESP32. Los archivos HTML, CSS y JavaScript se embeben directamente en el firmware usando `MBED_TXTFILES`. De esta manera, el ESP32 puede servir la interfaz desde flash sin depender de un sistema de archivos externo.

```
extern const unsigned char index_html_start[] asm("_binary_index_html_start");
extern const unsigned char index_html_end[]   asm("_binary_index_html_end");
```

Listing 6: Símbolos generados para archivos embebidos.

Método	Endpoint	Descripción
GET	/	Sirve el portal de configuración WiFi.
GET	/style.css	Sirve la hoja de estilos embebida en firmware.
GET	/app.js	Sirve el JavaScript del portal.
POST	/api/wifi	Recibe SSID y contraseña. Llama a <code>wifi_manager_set_credentials</code> .

Cuadro 10: Endpoints del servidor HTTP embebido.

El formulario envía datos en formato `application/x-www-form-urlencoded`. Se implementó una función `url_decode` propia para decodificar caracteres especiales sin agregar dependencias externas.

5.4. LCD Port

`lcd_port` es la capa de hardware de la pantalla. Inicializa el bus SPI, el controlador ST7789, el backlight, LVGL, los buffers de render y la tarea encargada de procesar LVGL.

La función principal es `lcd_port_init`. Esta función se llama una sola vez desde `app_main`, antes de cualquier otro módulo que use la pantalla. El flujo de inicialización es:

1. Crear el mutex que protege el acceso a LVGL.
2. Configurar el backlight, arrancando apagado y encendiéndolo al final.
3. Inicializar el bus SPI2 con DMA automático.
4. Crear e inicializar el panel ST7789 mediante `esp_lcd`.
5. Inicializar LVGL y crear el display.
6. Reservar un buffer parcial de 20 líneas en memoria DMA.
7. Registrar callbacks de `flush` y de `flush-ready`.
8. Instalar el tick periódico de LVGL.
9. Crear la tarea `lcd_lvgl_task`.

```
lv_init();
lv_display_t *display = lv_display_create(LCD_H_RES, LCD_V_RES);
void *draw_buffer = spi_bus_dma_memory_alloc(LCD_HOST, draw_buffer_size, 0);
lv_display_set_buffers(display, draw_buffer, NULL,
    draw_buffer_size, LV_DISPLAY_RENDER_MODE_PARTIAL);
```

Listing 7: Inicialización general de LVGL y buffer parcial.

El callback `lcd_lvgl_flush_cb` se ejecuta cuando LVGL quiere enviar un área renderizada al panel. El envío SPI es asíncronico, por lo que LVGL no puede reutilizar el buffer hasta que el callback `lcd_notify_lvgl_flush_ready` confirme que la transferencia terminó.

5.5. LCD Manager

`lcd_manager` es la capa de presentación. Trabaja con objetos LVGL de alto nivel, sin conocer pines, SPI ni detalles del ST7789. Esta separación permite cambiar el hardware de la LCD sin tocar la lógica de presentación, y viceversa.

El manager crea una estructura de pantalla fija: un título y tres líneas de contenido. Todas las pantallas reutilizan esos labels, cambiando solo el texto.

```
s_title = lcd_create_label(screen, 220, NULL);
s_line1 = lcd_create_label(screen, 220, NULL);
s_line2 = lcd_create_label(screen, 220, NULL);
s_line3 = lcd_create_label(screen, 220, NULL);
```

Listing 8: Estructura base de labels en LCD Manager.

Función	Qué muestra y cuándo
<code>lcd_show_waiting</code>	Pantalla inicial: control de stock, esperando QR.
<code>lcd_show_product</code>	Nombre, stock actual y cantidad a agregar. Se actualiza con cada pulsación.
<code>lcd_show_added</code>	Confirmación con nuevo stock después de actualizar.
<code>lcd_show_cancelled</code>	Mensaje de cancelación. No se modifica el stock.
<code>lcd_show_error</code>	Mensaje de error específico.

Cuadro 11: Pantallas principales implementadas en LCD Manager.

6. QR Handler, cámara y touchpad

6.1. QR Handler y cámara

El módulo `qr_handler` es responsable de capturar imágenes con la cámara OV2640 integrada en la placa ESP32-S2 y decodificar códigos QR en tiempo real. Su función principal es detectar códigos QR válidos, extraer la información estructurada del producto y notificar al núcleo del sistema mediante un callback.

El módulo actúa como productor de eventos: no modifica directamente `product_db`, no escribe en NVS y no decide transiciones de la FSM. Solo captura, decodifica y entrega datos limpios al resto del sistema.

6.2. Configuración de hardware de cámara

La configuración de la cámara fue optimizada para lectura de QR en condiciones de iluminación variable y con recursos limitados del ESP32-S2. Los parámetros principales son:

- Formato de imagen: `PIXFORMAT_GRAYSCALE`.
- Resolución: `FRAME_SIZE_QVGA`, equivalente a 160×120 píxeles.
- Framebuffer: `fb_count = 1`, ubicado en PSRAM.
- Modo de captura: `CAMERA_GRAB_LATEST`.
- Frecuencia XCLK: 10 MHz.

Señal	GPIO asociado
XCLK	GPIO 1.
SCCB/I2C	SDA GPIO 8, SCL GPIO 7.
Datos 8-bit	D0 a D7: GPIO 46, 45, 41, 39, 40, 38, 42 y 21.
Sincronización	VSYNC GPIO 2, HREF GPIO 3, PCLK GPIO 33.

Cuadro 12: Pines principales utilizados por la cámara.

Después de inicializar la cámara se aplican ajustes del sensor, como contraste, brillo, saturación y nitidez, para mejorar la legibilidad de los QR.

6.3. Flujo de la tarea de escaneo

La tarea `qr_scan_task` corre en un bucle infinito. Su stack es mayor que el de tareas simples porque el procesamiento con `quirc` necesita memoria adicional.

El flujo de trabajo es:

1. Capturar un frame con `esp_camera_fb_get`.
2. Preparar los buffers internos de `quirc` mediante `quirc_resize`.
3. Copiar el framebuffer en escala de grises usando `quirc_begin`, `memcpy` y `quirc_end`.
4. Detectar y extraer QRs con `quirc_count`, `quirc_extract` y `quirc_decode`.
5. Parsear el payload, separando id y nombre del producto.
6. Invocar el callback registrado con los datos limpios.
7. Aplicar un cooldown de 3000 ms tras una lectura exitosa para evitar duplicados.
8. Devolver siempre el framebuffer con `esp_camera_fb_return`.

El formato esperado del payload es del estilo `ID|Nombre`. Si el QR está corrupto o no se puede decodificar, se ignora sin modificar la base ni generar cambios de stock.

6.4. Touchpad

El módulo `touchpad` permite al usuario interactuar con las solicitudes mostradas en la LCD. Su responsabilidad consiste en inicializar el hardware, calibrar la capacitancia base, definir umbrales de detección y supervisar continuamente el estado de los botones.

Aunque el touchpad ESP-LyraP-TouchA v1.1 dispone de seis botones, se utilizaron cuatro. Los botones `PHOTO` y `NETWORK`, correspondientes a ciertos pines conflictivos con la LCD,

fueron omitidos para evitar interferencias. Quedaron funcionales VOL_UP, PLAY/PAUSE, VOL_DOWN y RECORD.

6.5. Calibración y detección de toques

El touchpad no utiliza umbrales fijos. Durante la inicialización, `touchpad_initial_scanning` realiza tres escaneos de dos segundos por cada botón. A partir de esos valores se calcula un benchmark, que representa el valor del sensor sin tocar. Luego se calcula un threshold como una variación del 5% respecto de ese benchmark.

La función `touchpad_is_pressed` compara el valor filtrado actual con el benchmark. Si la diferencia supera el threshold, devuelve `true`; en caso contrario, devuelve `false`. Esta calibración automática compensa pequeñas variaciones de temperatura, humedad o tolerancias del sensor.

6.6. Tarea de monitoreo e integración con la FSM

El monitoreo continuo se realiza mediante `touchpad_task`, iniciada por `touchpad_task_start`. Antes de crear la tarea se verifica que el touchpad esté inicializado, que exista `fsm_event_queue` y que la tarea no haya sido creada previamente.

La tarea consulta el estado de los botones cada 50 ms. Para evitar múltiples eventos al mantener un botón presionado, usa detección por flanco ascendente mediante una variable `was_pressed`. Solo se envía un evento cuando el estado cambia de no presionado a presionado.

Botón	Evento FSM	Uso principal
VOL_UP	EV_BTN_UP	Aumentar cantidad o desplazarse hacia arriba.
VOL_DOWN	EV_BTN_DOWN	Disminuir cantidad o desplazarse hacia abajo.
PLAY/PAUSE	EV_BTN_CONFIRM	Confirmar operación.
RECORD	EV_BTN_CANCEL	Cancelar operación.

Cuadro 13: Correspondencia entre botones del touchpad y eventos de la FSM.

7. Dashboard y telemetría

Aunque el dashboard no quedó completamente cerrado durante esta etapa del proyecto, se definió con claridad la forma en que deben enviarse los datos para poder visualizarlos.

En ThingsBoard, el stock de cada producto debe llegar como telemetría numérica para que los widgets gráficos puedan construir series temporales.

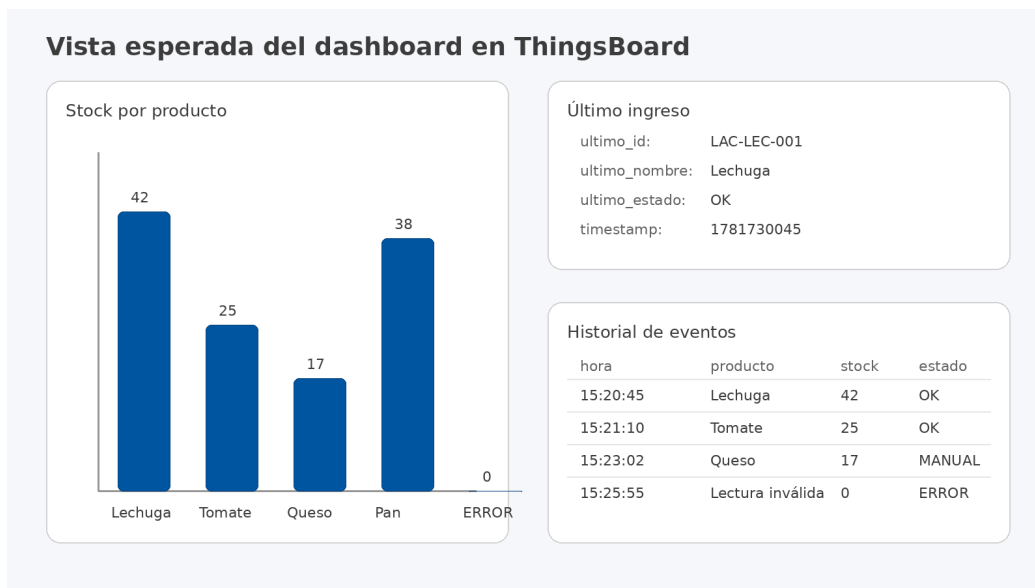


Figura 7: Vista esperada del dashboard para stock, último ingreso e historial de eventos.

Clave de telemetría	Tipo	Uso en dashboard
stock	Número	Stock general o último stock leído.
producto	Texto	Nombre del último producto procesado.
stock_lechuga	Número	Serie temporal del stock de lechuga.
stock_tomate	Número	Serie temporal del stock de tomate.
ultimo_ingreso	Número o fecha	Marca temporal del último evento.
estado	Texto	Estado del último evento: OK, MANUAL o ERROR.

Cuadro 14: Claves propuestas para visualización de stock e historial.

Inicialmente se pensó en utilizar dos brokers MQTT: uno en HiveMQ para la comunicación interna entre placas y otro en ThingsBoard para la telemetría. Sin embargo, esto aumentaba el riesgo de fallos y hacía más difícil diagnosticar problemas. Por eso se decidió usar un único broker en HiveMQ, con dos tópicos diferenciados: `/scan` para la comunicación entre placas y `/telemetry` para los datos consumidos por el dashboard.

Para mostrar correctamente el stock por producto, se cargaron inicialmente los productos existentes con stock en cero. Esta decisión evita problemas con widgets que no agregan dinámicamente nuevas series de datos.

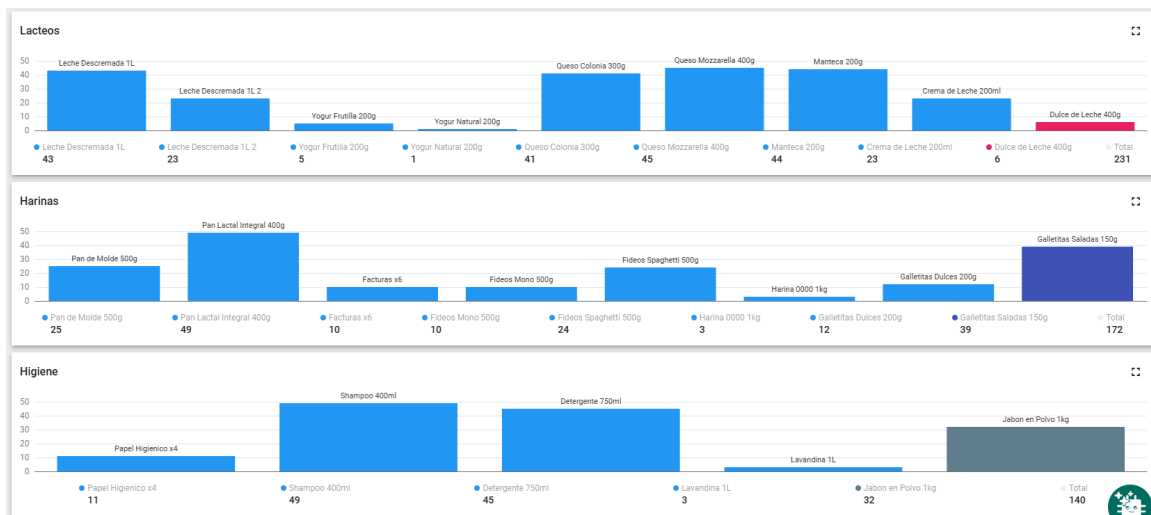


Figura 8: Primer conjunto de productos visualizados en el dashboard.



Figura 9: Continuación de productos visualizados en el dashboard.

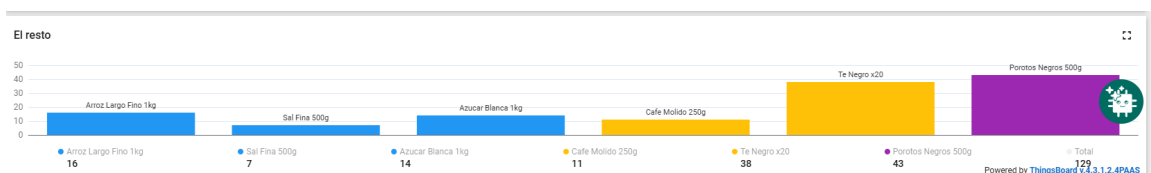


Figura 10: Último conjunto de productos visualizados en el dashboard.

8. Integración completa del sistema

8.1. Orden de inicialización

El orden de inicialización es importante porque varios módulos dependen de recursos creados por otros. Una secuencia general para la placa LCD es:

1. Crear `fsm_event_queue`.
2. Inicializar `lcd_port`: SPI, ST7789, LVGL y buffers.
3. Inicializar `lcd_manager`: labels y pantalla de espera.
4. Inicializar `product_db` y cargar el catálogo.
5. Inicializar la FSM y registrar callbacks.
6. Inicializar NVS una sola vez antes de los módulos que persisten datos.
7. Inicializar WiFi y HTTP para conectividad y configuración.
8. Inicializar logger, NTP y MQTT.
9. Inicializar touchpad y su tarea de monitoreo.

En la placa cámara, el flujo es más simple. Se inicializan red, NVS, pending queue, WiFi y MQTT antes de iniciar la cámara. Luego, cada QR detectado se publica por MQTT o se guarda como pendiente si la publicación falla.

8.2. Transcurso general del código

El recorrido integrado puede resumirse así:

1. El ESP32 arranca y crea la cola de eventos.
2. La LCD inicializa SPI, ST7789, LVGL y sus labels.
3. La pantalla muestra *Esperando QR* desde el inicio.
4. La FSM se inicializa con callbacks de LCD, MQTT y persistencia.
5. NVS carga credenciales WiFi, historiales y pendientes si existen.
6. El WiFi Manager intenta conectar como STA y levanta su AP de configuración.
7. Si se obtiene IP, se envía `EV_WIFI_CONNECT_SUCCESS`.
8. MQTT se conecta, se suscribe a los tópicos necesarios y puede reenviar pendientes.
9. La cámara escanea un QR y publica el producto por MQTT.
10. La placa LCD recibe el QR, llama a `fsm_on_qr_detected` y la FSM busca el producto.
11. La LCD muestra nombre, stock actual y cantidad a agregar.
12. El usuario ajusta la cantidad con el touchpad.
13. Al confirmar, la FSM actualiza `product_db`.
14. Se muestra el nuevo stock y se publica la telemetría.
15. El sistema vuelve a `STATE_IDLE`.

9. Pruebas y validación

Las pruebas se plantearon por niveles, desde componentes aislados hasta el flujo integrado entre placas.

Prueba	Resultado esperado
Inicialización de NVS	La partición se inicializa y se abre el namespace sin error.
Inicialización del logger vacío	Si no hay datos previos, head , tail y count quedan en cero.
Persistencia del logger	Luego de reiniciar, se recuperan buffer e índices.
Sincronización NTP	<code>get_timestamp</code> devuelve true y entrega un <code>time_t</code> válido.
Publicación MQTT	El broker recibe el JSON en el tópico correcto.
Recepción MQTT	La placa suscripta reconstruye el Product y lo guarda como recibido.
Filtro por device_id	Un mensaje propio no se guarda como recibido.
Error de formato JSON	El mensaje inválido se descarta sin romper el programa.
Producto encontrado	<code>product_db_find_by_id</code> devuelve el producto correcto.
Actualización de stock	<code>product_db_add_stock</code> suma la cantidad seleccionada.
FSM: QR válido	La FSM recorre validación, búsqueda, confirmación, actualización y publicación.
FSM: QR inválido	La FSM muestra error y vuelve a STATE_IDLE .
Touchpad	Cada pulsación genera un único evento por flanco ascendente.

Cuadro 15: Plan de pruebas general del sistema.

Para validar la FSM se implementaron pruebas de transición en `fsm_transition_test.c`. Estas pruebas inicializan la FSM, desactivan eventos automáticos con `fsm_set_auto_events_enabled(false)` y ejecutan secuencias controladas de eventos. Esto permite verificar la lógica de estados sin depender de cámara, LCD, touchpad o MQTT real.

10. Problemas encontrados y decisiones tomadas

Durante el desarrollo surgieron varias decisiones de diseño.

En primer lugar, se decidió enviar información en formato JSON en lugar de una cadena plana. Esto permitió incluir en un mismo mensaje el id del producto, nombre, stock, timestamp, estado y `device_id`. Además, al tener campos separados, el mensaje resulta más fácil de procesar en ThingsBoard.

También se decidió representar la hora como `time_t` y no como texto, porque ese formato facilita la comparación, ordenación y publicación de eventos. El campo `device_id` fue necesario para filtrar mensajes propios y evitar duplicados en el logger.

Otra decisión importante fue separar claramente la placa cámara de la placa LCD. La cámara no levanta la FSM principal porque no tiene interfaz de usuario ni debe modificar stock. La placa LCD concentra la lógica de confirmación y actualización del inventario.

En la FSM, se optó por usar callbacks para desacoplarla del hardware y de MQTT. Esto permitió probar la lógica antes de tener todos los módulos funcionando y evitó dependencias directas entre componentes.

En la base de datos, se eligió una tabla hash de tamaño fijo. Esta solución es eficiente para un catálogo acotado, no usa memoria dinámica y reduce el riesgo de fragmentación.

Finalmente, se incorporó NVS como capa común de persistencia. En lugar de que cada módulo manipule directamente la flash de forma distinta, todos utilizan funciones comunes como `nvs_storage_set_str`, `nvs_storage_set_int` y `nvs_storage_set_blob`.

11. Conclusiones

Se implementó una base funcional para un sistema de control de stock distribuido en dos placas ESP32-S2. La comunicación MQTT permite que la placa cámara envíe productos escaneados y que la placa LCD procese esos datos, los muestre al usuario, actualice el stock y publique telemetría.

La FSM organiza el comportamiento completo de la placa LCD y evita que la lógica del sistema quede distribuida en callbacks o módulos periféricos. `product_db` concentra las operaciones sobre productos y `hash_table` permite búsquedas rápidas sin memoria dinámica. NVS aporta persistencia para credenciales, loggers y eventos pendientes.

La incorporación del logger circular resolvió la necesidad de conservar un historial acotado sin consumir memoria de forma indefinida. Al integrarlo con NVS, ese historial deja de depender exclusivamente de la RAM y puede mantenerse luego de reinicios.

El módulo WiFi/HTTP permite configurar la red sin recompilar firmware, mientras que LCD Manager y Touchpad completan la interacción local. QR Handler y la cámara integran la captura real del producto, cerrando el flujo físico del sistema.

En conjunto, la arquitectura resultante es modular, clara y escalable. Cada componente tiene una responsabilidad concreta y se comunica con los demás mediante estructuras compartidas, colas, eventos y callbacks.

12. Trabajo futuro

Como trabajo futuro se propone:

- Terminar de validar la publicación definitiva hacia ThingsBoard.
- Normalizar las claves de telemetría para todos los productos.
- Mejorar el reenvío automático de eventos pendientes cuando vuelve la conexión MQTT.
- Realizar pruebas completas con dos placas reales operando simultáneamente durante tiempos prolongados.
- Completar la interfaz de historial en el dashboard.
- Agregar manejo más completo de productos fuera del catálogo si el alcance del proyecto lo requiere.
- Incorporar más pruebas unitarias para `product_db`, `hash_table`, `pending_queue` y FSM.

13. Referencias

- Espressif Systems. ESP-IDF Programming Guide: MQTT Client. <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/protocols/mqtt.html>
- Espressif Systems. ESP-IDF Programming Guide: Non-Volatile Storage Library. https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/storage/nvs_flash.html
- Espressif Systems. ESP-IDF Programming Guide: System Time and SNTP. https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/system_time.html
- Embedded Artistry. Creating a circular buffer in C and C++. <https://embeddedartistry.com/blog/2017/05/17/creating-a-circular-buffer-in-c-and-c/>
- ThingsBoard. Telemetry data collection. <https://thingsboard.io/docs/pe/user-guide/telemetry/>

A. Anexo: funciones principales

Función	Módulo	Descripción
<code>mqtt_handler_set_loggers</code>	MQTT	Guarda punteros a logger local y logger recibido.
<code>iniciar_mqtt</code>	MQTT	Configura cliente MQTT, registra manejador e inicia conexión.
<code>procesar_y_publicar</code>	MQTT	Publica un producto válido.
<code>procesar_y_publicar_error</code>	MQTT	Publica un evento de error usando una estructura <code>Product</code> .
<code>publicar_evento</code>	MQTT	Obtiene timestamp, guarda en logger, arma JSON y publica.
<code>procesar_json_recibido</code>	MQTT	Parsea JSON, valida campos, filtra mensajes propios y reconstruye producto.
<code>logger_init</code>	Logger	Inicializa logger y recupera estado desde NVS si existe.
<code>logger_push</code>	Logger	Inserta evento en buffer circular y guarda estado en NVS.
<code>init_time</code>	NTP	Inicializa SNTP y configura zona horaria.
<code>get_timestamp</code>	NTP	Obtiene hora actual en formato <code>time_t</code> .
<code>fsm_init</code>	FSM	Inicializa estado interno de la máquina.
<code>fsm_register_callbacks</code>	FSM	Registra funciones externas de LCD, MQTT y persistencia.
<code>fsm_on_qr_detected</code>	FSM	Informa a la FSM que llegó un QR decodificado.
<code>fsm_execute_transition</code>	FSM	Evalúa evento actual y ejecuta transición asociada.
<code>product_db_init</code>	<code>product_db</code>	Inicializa la base local sobre la tabla hash.
<code>product_db_find_by_id</code>	<code>product_db</code>	Busca producto por identificador.

Función	Módulo	Descripción
product_db_add_stock	product_db	Suma cantidad al stock de un producto.
hash_table_insert	hash_table	Inserta o actualiza una entrada en la tabla.
hash_table_find	hash_table	Busca una entrada por id.
nvs_storage_init	NVS	Inicializa almacenamiento persistente.
nvs_storage_set_blob	NVS	Guarda estructuras completas como blob.
nvs_storage_get_blob	NVS	Recupera estructuras completas desde NVS.
wifi_manager_init	WiFi	Inicializa AP+STA y lee credenciales guardadas.
http_handler_start	HTTP	Inicia servidor web de configuración.
lcd_port_init	LCD Port	Inicializa SPI, ST7789, LVGL y buffers.
lcd_manager_init	LCD Manager	Crea objetos LVGL reutilizables.
qr_handler_init	QR	Inicializa cámara y lanza tarea de escaneo.
touchpad_init	Touchpad	Inicializa sensores táctiles y calibración.
touchpad_task_start	Touchpad	Crea tarea de monitoreo y envío de eventos.

Cuadro 16: Resumen de funciones principales del sistema.